

UNIVERSITÀ DEGLI STUDI DI
MILANO-BICOCCA

ADVANCED MACHINE LEARNING
FINAL PROJECT

Social Media Extremism Detection

Authors:

Ruben Vandamme - 936753- r.vandamme@campus.unimib.it
Muhamad Khatib - 941094- m.khatib@campus.unimib.it

January 30, 2026



Abstract

This study evaluates BERT-style transformers (BERT, RoBERTa, HateBERT, and BERTweet) for detecting extremist content on social media. Experiments varying head architectures, pooling, learning rates, dropout, and data augmentation show that transformer choice drives performance, while other modifications offer marginal gains. Targeted augmentation improves results only slightly, highlighting the need for more diverse strategies. Using 5-fold cross-validation and mean-pooling, the final model reached 0.7333 accuracy on the Kaggle private test set. For small, specialized datasets, data quality and diversity outweigh incremental model tweaks; future work should explore richer augmentation or ensembles.

1 Introduction

The Social Media Extremism Detection Challenge [1] addresses the need to automate detection of radical content on social media, where manual moderation is infeasible. Exposure to extremist content can foster radicalization and threaten public safety, but detection is nuanced—it must distinguish harmful group-targeted hate from personal insults without infringing on free speech. In this study, we hypothesized that a BERT-based transformer with a small classifier head could capture complex linguistic patterns to achieve reasonable accuracy. We focus on experimenting with this setup to understand factors influencing prediction performance, contributing to broader efforts in social awareness and public good.

2 Datasets

The dataset provided by the challenge contains 2250 short, English-language social media messages labeled for the presence (1150) or absence (1100) of clearly extremist content.

Text	Label
well whatever do welcome immigrants to this country but whatever do not want those wishing to kill us and change our culture	EXTREMIST
you know how funny it is when an immigrant advocate tells you to go back home just because they do not like a conservative immigrant	NON_EXTREMIST

Table 1: Dataset example (extremist/non-extremist)

While not mentioned anywhere, it seems this dataset was already normalized beforehand including case folding, punctuation stripping and de-accenting. Three additional types of preprocessing were tested (see methodological approach):

1. Backtranslation: Using Helsinki-NLP/opus-mt-en-de, each sentence is translated to German and back to English. This preserves meaning while altering wording and structure, helping the model generalize extremist concepts instead of memorizing catchphrases.
2. Named Entity Recognition (NER) Swapping: Using spaCy’s en-core-web-sm, NORP (nationalities, religious, political groups) and GPE (countries/cities) entities are replaced with others from the same category. This teaches the model to recognize patterns of extremist rhetoric, like dehumanization or calls to violence, rather than specific names.
3. Bidirectional Adversarial Smart Swapping: Targeted entities are swapped with personal insults (and vice versa) while flipping the label. This creates near-identical sentence pairs with opposite classes, preventing over-reliance on specific names.

Regarding the BERT-style transformers, the ‘transformers’ package is used to automatically pull models from the Hugging Face Hub. Four different flavours were tested (see methodological approach).

1. BERT (bert-base-uncased) is a general-purpose model that learns word meanings from context, capturing subtle semantic patterns that may reveal implicit extremist language.
2. RoBERTa (roberta-base) is an optimized BERT trained on more data with better strategies, improving its ability to model complex or implicit rhetoric.

3. HateBERT (Hate-speech-CNERG/dehatebert-mono-english) is pre-trained on hate-speech content, enhancing recognition of aggressive, dehumanizing, or ideologically charged language.
4. BERTweet (vinai/bertweet-base) is trained on Twitter data, handling social media slang and informal language, making it well-suited for short, noisy posts containing extremist content.

3 The Methodological Approach

The base model consists of a BERT-style transformer body with a small linear classifier head. The head is trained from scratch, while the pretrained body maps contextual embeddings to EXTREMIST/NON-EXTREMIST labels. Binary Cross Entropy is used as the loss, and AdamW as the optimizer, with a low learning rate of 1e-5 to fine-tune the body slowly and avoid destroying pretrained knowledge. Overfitting is a risk due to the small dataset and millions of parameters, so weight decay and early stopping (patience = 3 epochs) are applied, and only the model with lowest validation loss is saved.

Initial runs with a frozen body yielded about 60% accuracy, highlighting the need to unfreeze the transformer to allow gradients to update its weights. Subsequent experiments modified single components of this base setup, summarized in Table 2.

It is important to note that the training and validation data were not fully representative of the Kaggle test set. Semantic patterns in the private test split differ from the public and training data, likely to discourage overfitting and reward true generalization.

To reflect this distinction, three evaluation metrics are reported:

1. Validation loss, which proved more stable and more representative of test performance than validation accuracy.
2. Kaggle public accuracy.
3. Kaggle private accuracy, considered the primary metric. The best configuration was selected based on this score, provided that training and public accuracy remained bounded to avoid overfitting.

3.1 Experiment 1: model selection

Four transformer models were evaluated: BERT, RoBERTa, HateBERT, and BERTweet. RoBERTa and BERTweet achieved the highest Kaggle private scores, followed by BERT. HateBERT performed considerably worse, likely because it was pretrained on hate speech content, which focuses on offensive language patterns and does not fully cover the broader linguistic forms found in extremist posts. These results suggest that general-purpose or social-media-specific transformers better capture the diversity of language in the dataset.

3.2 Experiment 2: head classifier structure

The choice of head architecture was tested to understand how much additional learning capacity is needed on top of a pretrained transformer body. The hypothesis was that a moderately sized bottleneck layer would balance learning capacity and regularization, potentially outperforming both a minimalist head and a large projection layer.

Three head designs were evaluated:

- **Minimalist:** Directly maps the transformer output to the classification label. This design relies entirely on the features provided by the body, providing no additional capacity for learning complex relationships.
- **Bottleneck:** Reduces the 768-dimensional transformer output to a smaller hidden layer (64 units) before classification, with a Tanh activation. This introduces some non-linear learning capacity while limiting overfitting.
- **Projection layer:** Maintains the full 768-dimensional output in an additional hidden layer, giving maximum capacity but also higher risk of overfitting, particularly for a small dataset.

Results showed that both the bottleneck and minimalist heads achieved similar performance, with the projection layer providing no meaningful improvement. The conclusion is that the transformer body captures the majority of the task-relevant features, and even a simple head is sufficient for classification. This confirms the hypothesis that additional complexity in the head is not necessary for this dataset.

3.3 Experiment 3: pooling strategies

This experiment tested different strategies for converting the transformer’s token-level outputs into a single sentence-level representation. The hypothesis was that mean pooling over all token embeddings might provide a more stable and informative representation than relying solely on the CLS token [2], especially when working with short posts that contain dispersed signals. At the layer level, attention-based pooling over multiple layers was compared with using only the final layer to determine whether the model could learn which layers contribute most to classification.

The strategies evaluated were:

- **CLS pooling:** Uses the default summary token ([CLS]) as the sentence representation, which is simple and efficient but may ignore relevant information from other tokens.
- **Mean pooling:** Averages all token embeddings, providing a representation that reflects contributions from the entire sentence. This approach is widely used in practice and can stabilize fine-tuning on smaller datasets.
- **Final-layer pooling:** Uses only the last transformer layer’s outputs, which is standard and computationally efficient.
- **Layer attention:** Aggregates selected upper layers using a learned attention mechanism, allowing the model to weight more informative layers higher.

Mean pooling tended to outperform CLS pooling, and attention-based layer aggregation offered a small improvement over using only the final layer. While the performance differences were modest, the experiment provided insight into how different token and layer pooling strategies influence transformer-based sentence representations and which configurations are more robust for classification.

3.4 Experiment 4: dropout regulation

Dropout was evaluated as a simple regularization technique to mitigate overfitting, which is a major concern when fine-tuning large pretrained models

on small datasets. Introducing a moderate dropout rate led to a measurable improvement in test accuracy (dropout 0.5 resulted in an increase of approximately 3%), indicating that regularization helped stabilize training and improve generalization.

3.5 Experiment 5: learning rate

In the base model, the same small learning rate was used for both the transformer body and the classification head. Variants were tested in which the head used a higher learning rate than the body, as well as configurations with a warmup schedule to gradually increase the body’s learning rate at the start of training. While these setups led to slower and more stable convergence, their impact on final accuracy was minimal. This suggests that using a single small learning rate for both components is sufficient for this dataset, and further reinforces the limited influence of the classification head.

An additional note: during development, the project initially used TensorFlow/Keras, but environment issues arose (Colab installs Keras 3 alongside TensorFlow 2.x, causing internal mismatches). A temporary bridge was implemented to propagate outputs between the transformer body and the head. However, now early stopping and saving the best model did not work reliably, partly because the Hugging Face transformers rely on PyTorch under the hood. Additionally, Keras’ compile and fit interface does not allow assigning different learning rates to the BERT body and the head. These limitations led to fully switching all code to PyTorch, which provided the required flexibility and removed the dependency on Keras/TensorFlow.

3.6 Experiment 6: data augmentation

Data augmentation techniques, including backtranslation, NER entity swapping, and adversarial smart swapping [3], were applied to increase the effective size of the training set and improve generalization. The expectation was that augmenting the dataset would help the model better handle variations in text and reduce overfitting.

These targeted augmentation methods showed only limited gains in accuracy, which was somewhat unexpected given the small size of the dataset. It seems a lot of care is needed while augmenting data. After online re-

search, studying other implementations [4], it was noticed they often used broader methods such as contextual word substitution, random insertion, swap, and deletion, and these yielded more substantial improvements. These findings suggest that while targeted entity-level augmentations help stabilize the model, introducing diverse lexical and structural variations is more effective for generalization.

3.7 Experiment 7: cross-validation

Cross-validation was applied to expose the model to multiple, slightly different views of the training data, rather than relying on a single split. This approach encourages the model to learn patterns that generalize across different subsets and reduces the risk of overfitting, particularly in noisy or imbalanced datasets.

Five-fold cross-validation was tested, with predictions combined using soft voting to produce the final output. Compared to training on a single split that might simply be lucky, this setup led to more stable performance across validation and test sets and showed a very clear and substantial improvement.

3.8 Experiment 8: weighted learning

Class imbalance in the test set (more extremist than non-extremist samples) was addressed by adjusting the loss weights during training, giving more importance to non-extremist examples. This helped the model avoid bias toward the majority class and can partially correct for distribution mismatches between training and evaluation sets.

4 Results and Evaluation

See Table 2 for a complete list of results for each experiment. These results should be interpreted with caution, as they are based on only a single seeded run.

After selecting the best feature from each experiment, the features were combined into a final configuration, shown in Code Snippet 1. The execution took

approximately an hour, largely due to the use of 5-fold cross-validation and the doubled backtranslated dataset. The final accuracy was 0.7333.

Table 2: Model performance results, including experiments.

Experiment Name	Validation Loss	Kaggle Public	Kaggle Private
s_RoBERTa	0.3848	0.8560	0.7360
s_HateBERT	0.4501	0.7840	0.6320
s_BERTTweet	0.4022	0.8293	0.7360
s_BERT	0.3614	0.8587	0.7253
s_projection_layer	0.3927	0.8347	0.7173
s_bottleneck	0.3663	0.8293	0.7360
s_minimalist	0.3687	0.8240	0.7227
s_mean_attn	0.3574	0.8427	0.7467
s_cls_attn	0.3815	0.8347	0.7227
s_mean_last	0.3585	0.8373	0.7307
s_cls_last	0.4022	0.8293	0.7360
s_8	0.3951	0.8480	0.7360
s_6	0.3684	0.8427	0.7520
s_5	0.3668	0.8560	0.7653
s_4	0.3631	0.8347	0.7520
s_2	0.3847	0.8347	0.7333
s_0	0.3663	0.8293	0.7333
s_warmup_and_decay	0.4159	0.8240	0.7333
s_separate_learning_rate	0.4150	0.8267	0.7227
s_global_learning_rate	0.4022	0.8293	0.7360
s_augment_use_all	–	0.7947	0.6933
s_adv_swap_test	–	0.8373	0.7173
s_adv_swap	–	0.7360	0.6133
s_ner	–	0.8400	0.7173
s_backtr	–	0.8453	0.7387
s_not_augmented	–	0.8293	0.7360
	0.4057		
	0.4097		
	0.3958		
	0.3587		
s_5_folds	0.3855	0.8507	0.7627
s_1_fold	0.4022	0.8293	0.7360
s_weight_non_extr_more	–	0.8347	0.7493
s_weight_extr_more	–	0.8133	0.7067
s_no_weight	–	0.8293	0.7360
ALL EXTREMIST (mostly non-extremist / slight non-extremist)		0.2240	0.4373
s_final	–	0.8400	0.7333

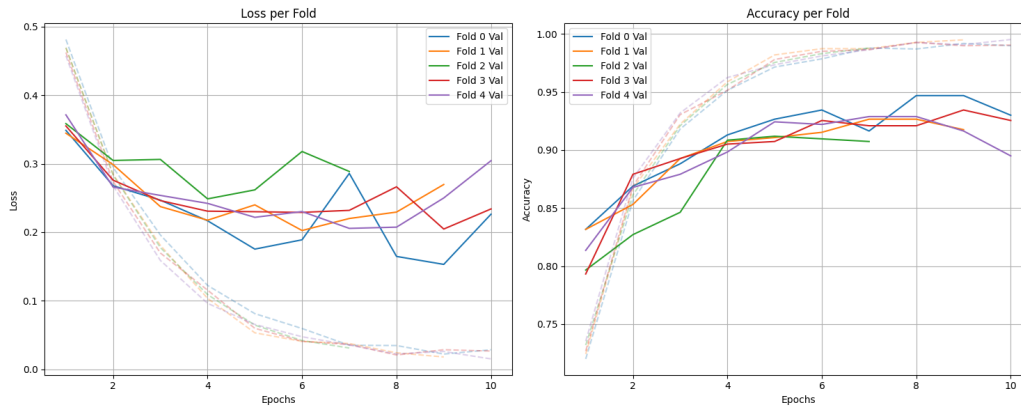


Figure 1: Training graph of the final combined model

Listing 1: Final model code snippet

```

1  try:
2      load_csv_and_augment(use_aug=['backtr'])
3      train(
4          'BERTweet',
5          'vinai/bertweet-base',
6          head=torch.nn.Sequential(
7              torch.nn.Dropout(0.5),
8              torch.nn.Linear(768, 1),
9              torch.nn.Sigmoid()
10         ),
11         token_pooling='mean',
12         layer_pooling='attn',
13         layers_to_use=[9, 10, 11],
14         n_splits=5,
15         pos_weight=0.8,
16         appendix='final',
17     )
18  except:
19      pass
20
21  # reset
22  load_csv_and_augment(use_aug=[])

```

5 Discussion

The final results were underwhelming: the accuracy reached only 0.7333, while the projected value was around 0.8. This means the model placed about 40th out of the 164 contestants. One factor that had not been fully

accounted for during earlier experiments is that individual changes can interact with each other in complex ways, which likely explains the discrepancy. Even after testing multiple configurations, the highest achievable accuracy was around 0.77, providing little improvement over single changes applied to the baseline model. This suggests though that, without more meaningful data augmentation, the maximum performance achievable by a BERT-style model with a classification head is around 0.8.

Areas for improvement to take into consideration next time include starting with k-fold cross-validation to obtain more stable results, combined with averaging over multiple runs with different random seeds. It would have become clear earlier that a BERT-style model on its own performs well, and that tweaking the classification head or applying additional tricks is unlikely to yield significant gains. Efforts would have been better directed toward improving the data, such as through meaningful data augmentation and optimizing its use with cross-validation. While this challenge emphasized accuracy, it is recommended to also consider metrics such as the F1 score to better balance coverage and confidence.

6 Conclusions

This study evaluated BERT-style transformers for detecting extremist content on social media. Experiments tested different model bodies, head architectures, pooling, dropout, learning rates, data augmentation, and cross-validation. Results show the transformer body is most critical: general-purpose and social-media-specific models outperform specialized hate-speech models. Head complexity, pooling, and learning rate tweaks offered minimal gains. Data augmentation and cross-validation helped slightly, but targeted augmentations alone were insufficient. Maximum accuracy with a simple head is about 0.8 without richer, more diverse data. Future work should focus on better training data or complementary model ensembles. Overall, data quality outweighs incremental model adjustments for small, specialized datasets.

References

- [1] “Social media extremism detection challenge,” Kaggle Competitions, 2024, <https://www.kaggle.com/competitions/social-media-extremism-detection-challenge/overview>.
- [2] Zilliz Cloud, “Why is mean pooling often used on the token outputs of a transformer (like bert) to produce a sentence embedding?” <https://blog.milvus.io/ai-quick-reference/why-is-mean-pooling-often-used-on-the-token-outputs-of-a-transformer-like-bert-to-produce-a-sentence-embedding>, 2026, accessed: 2026-01-??
- [3] G. D. DL, “Several text data augmentation methods in nlp,” Medium, 2025, <https://medium.com/@tubelwj/several-text-data-augmentation-methods-in-nlp-cc610135779b>.
- [4] “Social media extremism detection — 1st place solution,” Kaggle Competitions Write-ups, 2026, <https://www.kaggle.com/competitions/social-media-extremism-detection-challenge/writeups/social-media-extremism-detection-1st-place-solut>.